

# ORAN軟體模擬與xApp動態資源調配節能實作

「行動通訊主題式跨層次系統整合教學聯盟計畫」

D2 模組：「人工智慧於存取網路資源管理與節能技術應用」

主持人:國立臺北科技大學 電子工程學系 潘孟鉉教授

課程助教:謝欣蓉

# Contents

1. Introduction
2. OAI Setting Up (已實作)
3. RF Simulator + FlexRIC (已實作)
4. xApp Development (KPM\_Capture)(請實作其中的 xApp 邏輯)
5. xApp Development (KPM\_Capture)--測試結果
6. 本機驗證模型推論

# Introduction

# Open RAN 的興起背景

- Open RAN 是無線網路架構中日益受到重視的創新方案
- 傳統無線接入網(RAN)架構具有高度專有性與封閉性
- 行動網路營運商面臨供應商鎖定問題
- 缺乏彈性與多樣性開放架構被視為解決現有RAN限制的潛在方案
- Open RAN 允許多個供應商提供不同模組，提高互操作性與選擇性

# Open RAN 的核心概念

- Open RAN 提倡使用標準化與開放的接口連接各種網路模組
- 主要元件包括：分散式單元(DU)、中央單元(CU)與無線電單元(RU)
- 架構支援虛擬化部署，提升靈活性與擴展能力
- 不同廠商提供不同模組，可實現最佳化組合
- 目標在於實現跨供應商設備之間的互通與整合

# O-RAN 架構原則之一：虛擬化

- 虛擬化指的是將 RAN 所需的硬體與軟體解耦
- 能夠將 RAN 功能作為開放軟體，在通用硬體平台上執行
- 不需依賴專屬硬體，有助於使用具成本效益的最佳硬體組件
- 虛擬化使得資源能夠具備彈性擴展能力，以應對網路需求變化
- 分層抽象(layered abstraction)簡化 RAN 的協調管理
- 有潛力大幅降低營運成本

# O-RAN 架構原則之二：解耦與開放接口

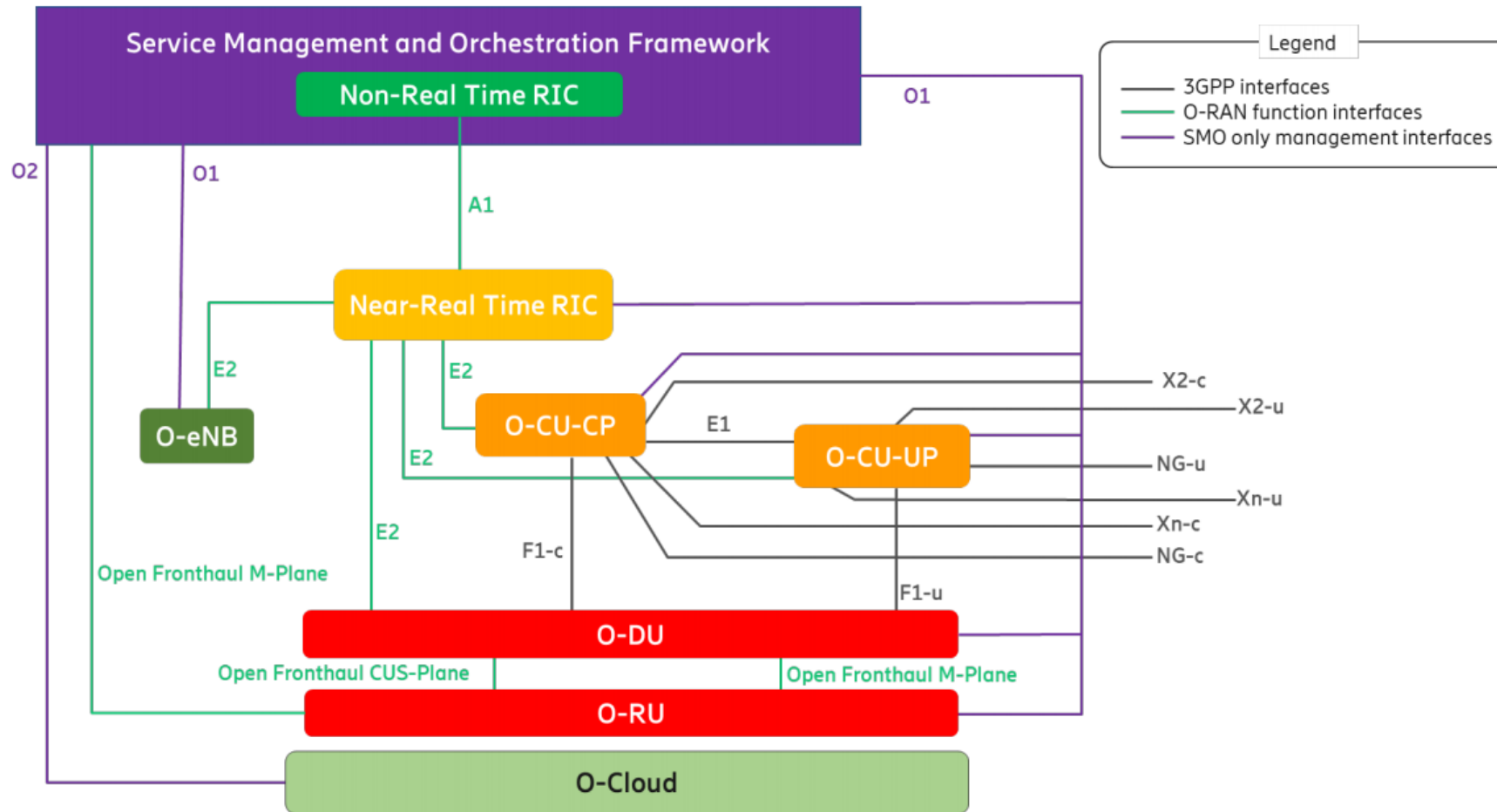
- 解耦(Disaggregation)可將基站(BS)分成多個功能單元
  - 在 5G 中，BS 通常分為三個邏輯單元：RU、DU、CU
  - CU 可再細分為 CU-CP(控制面)與 CU-UP(使用者面)
- 此種邏輯分離有助於在不同網路位置與硬體平台上部署各種功能
- 開放接口(Open Interfaces)是指 RAN 各元件之間的標準化接口
  - 允許不同供應商設備間互通 (interoperability)
  - 可根據效能、功能、時程與成本等條件進行最適設備組合配置

# O-RAN 架構原則之三：智慧控制

- RAN Intelligent Controller(RIC)是 O-RAN 架構中的關鍵元件
- RIC 提供平台以最佳化 RAN 元件與資源
- 分為兩種形式：
  - 非即時 RIC(Non-RT RIC)：負責非即時的控制與優化
  - 近即時 RIC(Near-RT RIC)：執行近即時控制與優化
- 這些 RIC 可作為 AI/ML 應用(如 rApps、xApps)的宿主平台
- AI/ML 應用可執行如無線資源管理(RRM)、自組織網路(SON)等任務
- 外部控制器的導入，有助於以更智慧、低成本的方式管理網路服務



# O-RAN 架構圖



# OpenAirInterface

- OpenAirInterface (OAI) 是一個開源的軟體平台，主要用於實作遵循 3GPP 標準的行動通訊系統（包含 4G LTE 與 5G NR）



# OpenAirInterface

- 核心架構:

OAI 提供了完整的 5G 端到端 (End-to-End) 軟體網路實作，主要包含三個核心部分：

1. 5G 無線接取網 (OAI-RAN)：實作了 5G 基地台 (gNB) 的功能，涵蓋底層的實體層 (PHY) 以及完整的上層協定疊 (MAC, RLC, PDCP, RRC, SDAP)。支援非獨立組網 (NSA) 與獨立組網 (SA) 模式。
2. 5G 核心網 (OAI-CN5G)：實作了 5G 獨立組網 (SA) 運作所需的網雲原生網路功能 (NF)，例如 AMF、SMF、UPF、NRF 與 UDM 等。這些組件高度支援容器化部署。
3. 使用者設備 (OAI-UE)：提供軟體定義的 5G 終端設備模擬，讓開發者在缺乏商用 5G 手機或實體終端模組的情況下，也能進行從終端到核心網的連線測試。

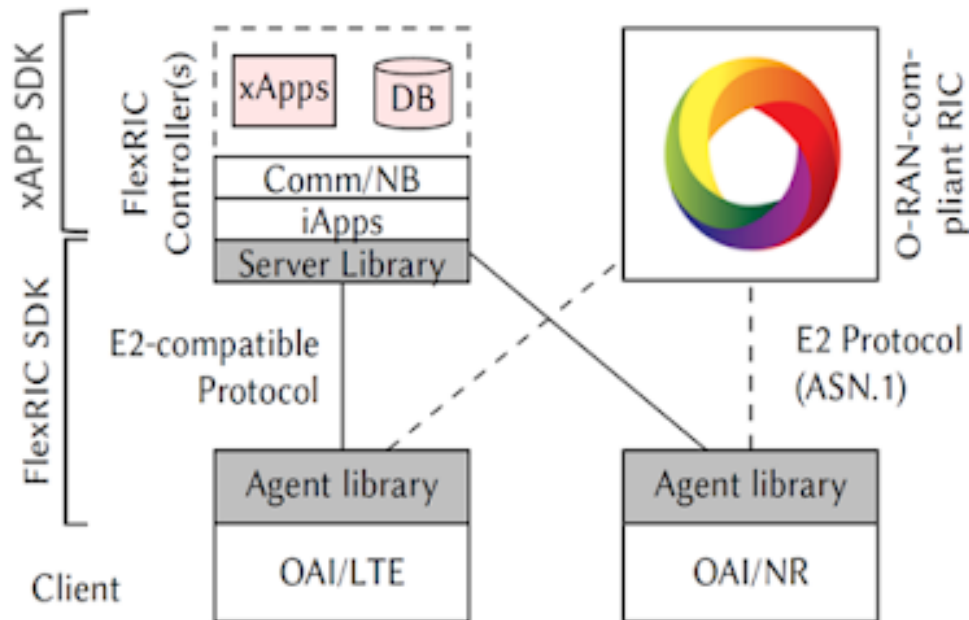
# OpenAirInterface

- 技術特色:
  1. 基於軟體定義無線電 (SDR)：OAI 將繁重的基頻訊號處理與通訊協定全部以軟體實作，並運行於一般的 Linux 主機上。實際的無線電波收發，則透過連接 SDR 硬體（如常見的 USRP 系列）來完成。
  2. 高度遵循 3GPP 標準：持續跟進 3GPP Release 15 與 Release 16 的標準規範，確保其開源基地台或核心網能夠與市面上的商用 5G 設備互相操作。
  3. 支援 O-RAN 開放架構：因應開放式無線接取網 (O-RAN) 的趨勢，OAI 的 RAN 架構支援 3GPP 的功能拆分 (如 CU 與 DU 分離架構、F1 介面)，並積極整合 O-RAN 聯盟的 E2 介面，使其成為學術界與產業界測試 RIC (RAN Intelligent Controller) 及開發 xApp 的重要開源基底。



# FlexRIC

- FlexRIC 是一個專為 O-RAN 設計的開源、彈性且高效能的 RAN 智慧控制器 (RAN Intelligent Controller, RIC)，在 O-RAN 架構中主要扮演近即時 (Near-Real-Time, Near-RT) RIC 的角色。
- FlexRIC 的設計目標是提供一個輕量且可高度客製化的平台，讓研究人員與開發者能夠輕易地測試與部署網路控制邏輯 (xApp)。



# FlexRIC

- 核心架構與運作機制:
  1. E2 介面實作：FlexRIC 完整實作了 O-RAN 聯盟定義的 E2 介面。它分為運行於 RAN 節點 (如 OAI 的 CU/DU) 內的 E2 代理程式 (E2 Agent)，以及運行於控制器端的 E2 伺服器 (E2 Server)。兩者透過 E2 介面進行通訊。
  2. E2 服務模型 (E2SM)：支援多種標準與自定義的 E2 服務模型。例如支援用於收集效能數據的 E2SM-KPM (Key Performance Measurement)，以及用於控制 RAN 行為的 E2SM-RC (RAN Control)，讓控制器能夠讀取基地台狀態並下達指令。
  3. xApp 執行環境：接收來自底層網路的數據，將其提供給 xApp 進行運算，再將 xApp 的決策轉化為控制訊息回傳給 RAN 節點。

# OAI Setting Up (已實作)

# Basic Hardware Requirement

No of PCs: 1 PC or 2 PCs.

Laptop/Desktop/Server for OAI CN5G . OAI gNB . OAI UE

- Operating System: [Ubuntu 22.04 LTS](#)
- CPU: 8 cores x86\_64 @ 3.5 GHz
- RAM: 8 GB

- Reference :

<https://github.com/OPENAIRINTERFACE/openairinterface5g/blob/develop/doc/README.md>



# OAI Setting Up

## Setup OAI CN5G

### Step 1:OAI CN5G Pre-Requisites

```
cd ~
```

```
# 安裝基礎網路與版本控制工具  
sudo apt install -y git net-tools putty
```

```
# 準備 Docker 的安裝環境  
sudo apt update
```

```
# 更新套件庫清單，並安裝 curl (用來從網路下載檔案) 與 ca-certificates (用來驗證安全的 SSL 憑證連線)  
sudo apt install -y ca-certificates curl
```

```
# 建立一個專門存放密碼金鑰的目錄 /etc/apt/keyrings，並設定適當的資料夾權限  
sudo install -m 0755 -d /etc/apt/keyrings
```

```
# 下載 Docker 官方的 GPG 驗證金鑰，確保後續下載的 Docker 軟體是官方提供且未經竄改的  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
```

# OAI Setting Up

# 更改檔案權限，確保系統上所有使用者都有權限讀取這個金鑰檔案

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

# 將 Docker 的官方軟體庫網址加入到系統的 apt 來源清單中，這樣系統就知道要去哪裡抓取最新版的 Docker

# (注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]  
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

```
lindor@lindor:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

# 再次更新套件庫清單，讓系統讀取剛剛加入的 Docker 軟體庫

```
sudo apt update
```

# 這行指令安裝了 Docker 引擎 (docker-ce)、命令列工具 (docker-ce-cli)、底層容器執行環境 (containerd.io) 以及 docker-compose-plugin。docker-compose-plugin 對 OAI 非常重要，因為 OAI CN5G 是透過 YAML 檔來一次啟動所有的網路節點容器

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

# 將你目前登入的使用者帳號加入到 docker 群組中

```
sudo usermod -a -G docker $(whoami)
```

```
reboot
```

# OAI Setting Up

## Step 2: OAI CN5G Configuration Files

# 下載 OAI CN5G 設定檔壓縮檔

# (注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)

```
wget -O ~/oai-cn5g.zip https://gitlab.eurecom.fr/oai/openairinterface5g/-/archive/develop/openairinterface5g-develop.zip?path=doc/tutorial_resources/oai-cn5g
```

```
lindor@lindor:~$ wget -O ~/oai-cn5g.zip https://gitlab.eurecom.fr/oai/openairinterface5g/-/archive/develop/openairinterface5g-develop.zip?path=doc/tutorial_resources/oai-cn5g
```

# 將剛剛下載的壓縮檔解壓縮到目前的目錄

```
unzip ~/oai-cn5g.zip
```

# 將 oai-cn5g 目錄，直接拉出來放在家目錄底下

```
mv ~/openairinterface5g-develop-doc-tutorial_resources-oai-cn5g/doc/tutorial_resources/oai-cn5g ~/oai-cn5g
```

# 清理暫存與多餘檔案

```
rm -r ~/openairinterface5g-develop-doc-tutorial_resources-oai-cn5g ~/oai-cn5g.zip
```

# OAI Setting Up

## Step 3: Pull OAI CN5G Docker Images

```
cd ~/oai-cn5g
```

```
# 下載核心網容器映像檔
docker compose pull
```

## Step 4: Test OAI CN5G

```
cd ~/oai-cn5g
```

```
# 在背景啟動 5G 核心網
docker compose up -d
```

```
# 檢查容器運行狀態
docker compose ps -a
```

```
lindor@lindor:~/oai-cn5g$ docker compose up -d
[+] up 10/10
Trash tainer oai-nrf      Running
✓ Container oai-ext-dn    Running
✓ Container mysql         Running
✓ Container ims           Running
✓ Container oai-udr       Running
✓ Container oai-udm       Running
✓ Container oai-ausf      Running
✓ Container oai-amf       Running
✓ Container oai-smf       Running
✓ Container oai-upf       Running
```

```
lindor@lindor:~/oai-cn5g$ docker compose ps -a
```

| NAME           | IMAGE                                  | COMMAND                  | SERVICE    | CREATED            | STATUS                        | PORTS                                   |
|----------------|----------------------------------------|--------------------------|------------|--------------------|-------------------------------|-----------------------------------------|
| ims            | oaisoftwarealliance/ims:latest         | "asterisk -fp"           | ims        | 2 minutes ago      | Up About a minute (healthy)   |                                         |
| mysql          | mysql:8.0                              | "docker-entrypoint.s..." | mysql      | 2 minutes ago      | Up About a minute (unhealthy) | 3306/tcp, 33060/tcp                     |
| oai-amf        | oaisoftwarealliance/oai-amf:develop    | "/openair-amf/bin/oa..." | oai-amf    | 2 minutes ago      | Up About a minute (healthy)   | 80/tcp, 5342-5344/tcp, 8080/tcp, 9090/t |
| cp, 38412/sctp |                                        |                          |            |                    |                               |                                         |
| oai-ausf       | oaisoftwarealliance/oai-ausf:develop   | "/openair-ausf/bin/o..." | oai-ausf   | 2 minutes ago      | Up About a minute (healthy)   | 80/tcp, 5342-5344/tcp, 8080/tcp         |
| oai-ext-dn     | oaisoftwarealliance/trf-gen-cn5g:jammy | "/bin/bash -c ' ip r..." | oai-ext-dn | 2 minutes ago      | Up About a minute (healthy)   |                                         |
| oai-nrf        | oaisoftwarealliance/oai-nrf:develop    | "/openair-nrf/bin/oa..." | oai-nrf    | 2 minutes ago      | Up About a minute (healthy)   | 80/tcp, 5342-5344/tcp, 8080/tcp         |
| oai-smf        | oaisoftwarealliance/oai-smf:develop    | "/openair-smf/bin/oa..." | oai-smf    | About a minute ago | Up About a minute (healthy)   | 80/tcp, 5342-5344/tcp, 8080/tcp, 9090/t |
| cp, 8805/udp   |                                        |                          |            |                    |                               |                                         |
| oai-udm        | oaisoftwarealliance/oai-udm:develop    | "/openair-udm/bin/oa..." | oai-udm    | 2 minutes ago      | Up About a minute (healthy)   | 80/tcp, 5342-5344/tcp, 8080/tcp         |
| oai-udr        | oaisoftwarealliance/oai-udr:develop    | "/openair-udr/bin/oa..." | oai-udr    | 2 minutes ago      | Up About a minute (healthy)   | 80/tcp, 8080/tcp                        |
| oai-upf        | oaisoftwarealliance/oai-upf:develop    | "sh /openair-upf/bin..." | oai-upf    | About a minute ago | Up About a minute (healthy)   | 2152/udp, 8805/udp, 5342-5344/tcp       |

# OAI Setting Up

- Setup OAI gNB

## Step 1: Build UHD From Source

cd ~

# 安裝了編譯 UHD 所需的所有基礎工具與函式庫

# (注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)

```
sudo apt install -y autoconf automake build-essential ccache cmake cpufrequtils doxygen ethtool g++ git inetutils-tools  
libboost-all-dev libncurses-dev libusb-1.0-0 libusb-1.0-0-dev libusb-dev python3-dev python3-mako python3-numpy  
python3-requests python3-scipy python3-setuptools python3-ruamel.yaml
```

# 下載原始碼並指定版本

```
git clone https://github.com/EttusResearch/uhd.git ~/uhd
```

```
cd ~/uhd
```

```
git checkout v4.8.0.0
```

# 建立一個獨立的 build 資料夾，可以避免編譯產生的暫存檔污染原始碼目錄

```
cd host
```

```
mkdir build
```

```
cd build
```

# OAI Setting Up

# 讀取上一層目錄的設定，產生對應 Linux 環境的 Makefile

```
cmake ../
```

# 將 C/C++ 程式碼編譯成執行檔與函式庫

```
make
```

```
make -j 4
```

# 執行內建的單元測試

```
make test
```

# 將編譯好的二進位檔案與函式庫安裝到 Linux 系統的全域目錄中

```
sudo make install
```

# 更新系統的動態連結庫快取

```
sudo ldconfig
```

# USRP 硬體內部包含 FPGA，需要特定的韌體映像檔才能正常開機與運作。這行指令會連線到 Ettus 伺服器，自動下載與剛安裝的 UHD v4.8.0.0 版本完全匹配的 FPGA 韌體檔案到主機中

```
sudo uhd_images_downloader
```

# OAI Setting Up

## Step 2: Build OAI gNB

# 回到家目錄

```
cd ~
```

# 取得 OAI 5G 原始碼與切換分支

```
git clone https://gitlab.eurecom.fr/oai/openairinterface5g.git ~/openairinterface5g
```

```
cd ~/openairinterface5g
```

```
git checkout 2026.w09
```

```
git submodule update --init --recursive --force
```

# 到編譯目錄

```
cd ~/openairinterface5g/cmake_targets
```

# 執行 OAI 提供的自動化建置腳本， -I 參數代表 Install dependencies

```
./build_oai -I
```

# RF Simulator + FlexRIC (已實作)



- Prerequisites

## Step 1: 安裝與設定 GCC 編譯器

# 更新系統的套件庫清單，並將已安裝的軟體升級到最新版本

```
sudo apt update -y
```

```
sudo apt upgrade -y
```

# 安裝 Linux 軟體開發的基礎套件包，包含 make 工具以及基礎的 C/C++ 編譯環境

```
sudo apt install -y build-essential
```

# 明確指定安裝第 13 版的 GCC (C 編譯器)、G++ (C++ 編譯器) 以及 CPP (C 預處理器)

```
sudo apt install -y gcc-13 g++-13 cpp-13
```

# 用來管理系統中多個版本的編譯器。將 gcc-13 註冊為系統 gcc 指令的候選版本，優先級設為 100，同時將 g++ 與 gcov 綁定為從屬項目，確保當切換 C 編譯器版本時，C++ 編譯器等相關工具也會自動同步切換到第 13 版

# (注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)

```
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-13 100 --slave /usr/bin/g++ g++ /usr/bin/g++-13 --slave /usr/bin/gcov gcov /usr/bin/gcov-13
```

# 終端機會跳出一個選單，可手動選擇系統預設要使用哪一個版本的 GCC。在此步驟輸入對應 GCC 13 的編號

```
sudo update-alternatives --config gcc
```

# 確認當前的編譯器版本確實已經成功切換為 13.x.x 版

```
gcc --version
```

```
g++ --version
```

- **Installation for FlexRIC and E2 Agent**

## **Step 1: 初始化 FlexRIC 子模組及編譯 FlexRIC**

```
cd ~/openairinterface5g/openair2/E2AP/flexric
```

# FlexRIC 專案內部還依賴了其他外部的開源程式碼庫 (例如特定版本的通訊協定庫)。這行指令會自動初始化並下載所有必要的附屬子模組，確保編譯時不會缺少檔案

```
git submodule init && git submodule update --recursive
```

```
mkdir -p build && cd build
```

# -G Ninja：使用 Ninja 取代 Make 以加速編譯。

# -DCMAKE\_BUILD\_TYPE=Release：以發布模式編譯，這會啟用程式碼最佳化，讓 RIC 執行效能更好。

# -DKPM\_VERSION=KPM\_V3\_00：指定使用 KPM 服務模型的第 3.00 版。這是 xApp 用來向基地台請求效能數據 (如 PRB 使用率、吞吐量) 的標準協定。

# -DE2AP\_VERSION=E2AP\_V2：指定使用 E2AP 的第 2.0 版。這是 RIC 與基地台之間建立連線的底層控制協定。

```
cmake -G Ninja -DCMAKE_BUILD_TYPE=Release -DKPM_VERSION=KPM_V3_00 -DE2AP_VERSION=E2AP_V2 ..
```

# 執行編譯，並將編譯出的 FlexRIC 函式庫安裝到系統路徑中

```
ninja
```

```
sudo ninja install
```

- **Installation for FlexRIC and E2 Agent**

## Step 2:編譯支援 E2 Agent 的 OAI RAN

```
cd ~/openairinterface5g
```

```
# 載入 OAI 的環境變數設定  
source oaienv
```

```
cd cmake_targets
```

# --build-e2：這個參數會啟用基地台內部的 E2 Agent 功能。沒有這個參數，gNB 就只是一個普通的 5G 基地台；加上之後，它就成為了一個相容 O-RAN 標準的 E2 節點。

# --cmake-opt ...：將剛剛在 FlexRIC 決定的 E2AP 與 KPM 版本參數傳遞給基地台的編譯器。RIC 與基地台兩邊使用的協定版本必須完全一致，否則後續 E2 Setup 連線會失敗。

# 適用於 USRP and Rfsimulator 上

# **(注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)**

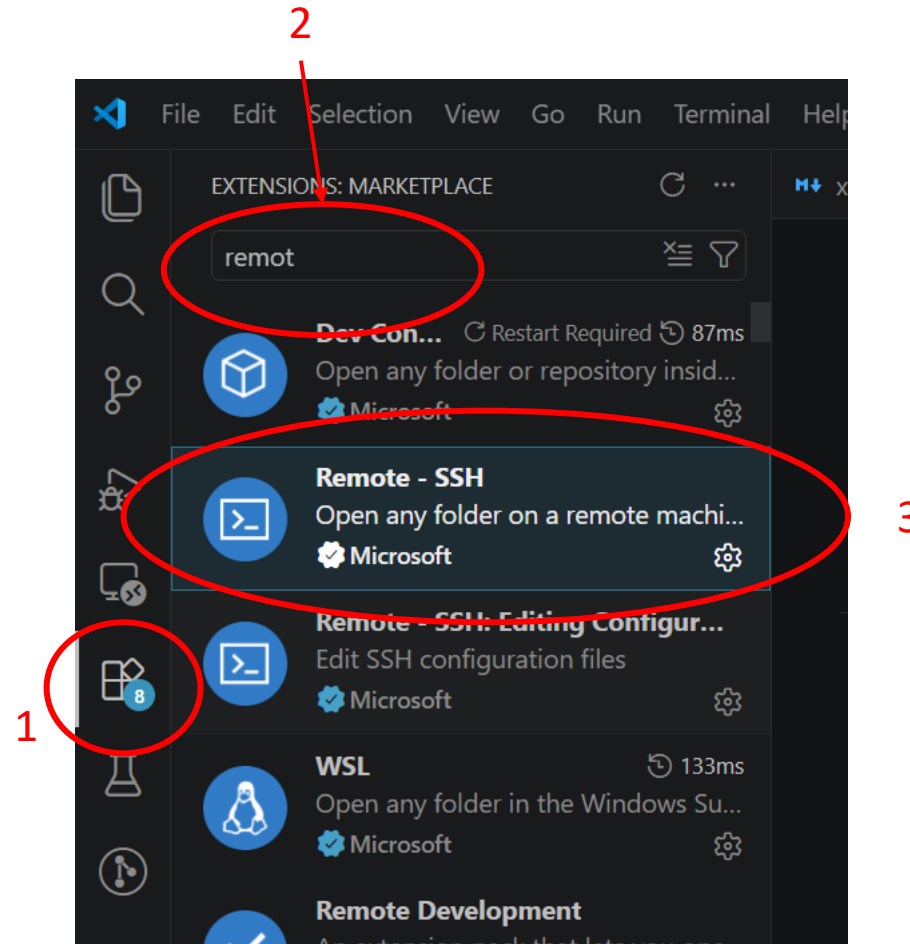
```
sudo ./build_oai --gNB --nrUE --build-e2 --ninja -w USRP -C --cmake-opt -DE2AP_VERSION=E2AP_V2 --cmake-opt -  
DKPM_VERSION=KPM_V3_00 --cmake-opt -DCMAKE_BUILD_TYPE=Release
```

```
lindor@lindor:~/openairinterface5g/cmake_targets$ sudo ./build_oai --gNB --nrUE --build-e2 --ninja -w USRP -C --cmake-opt -DE2AP_VERSION=E2AP_V2 --cmake-opt -DKPM_VERSION=KPM_V3_00 --c  
make-opt -DCMAKE_BUILD_TYPE=Release
```

# xApp Development (KPM\_Capture)

# VS Code Setting Up

安裝 Remote - SSH 擴充套件

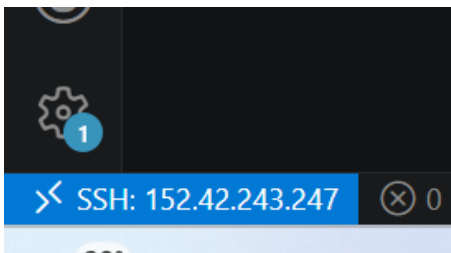


# VS Code Setting Up

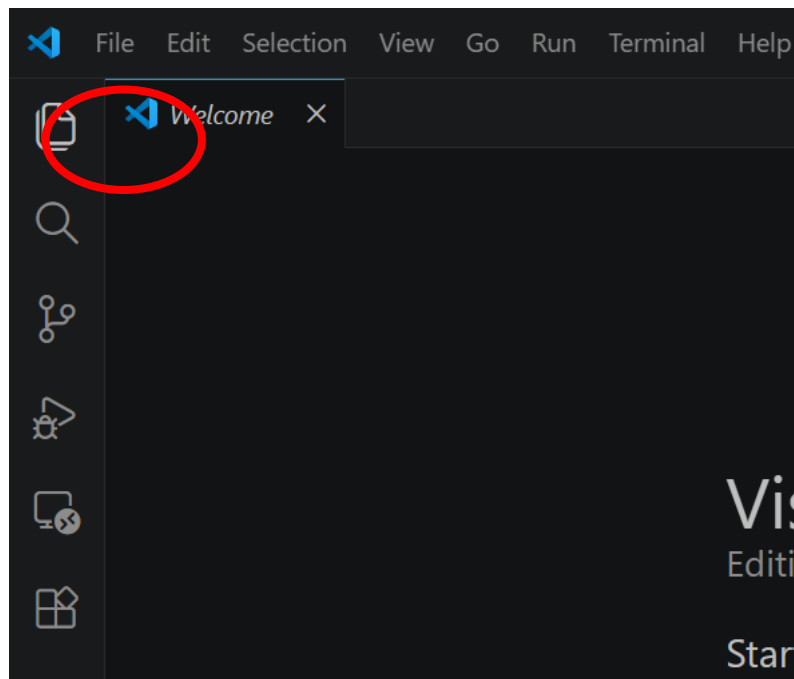
- 透過套件連線至 VM
  1. 安裝完成後，按下 F1 或 Ctrl+Shift+P 開啟命令選擇區
  2. 輸入並點選 Remote-SSH: Connect to Host... (連線至主機)
  3. 選擇 + Add New SSH Host... (新增 SSH 主機) 直接在輸入框打上你在終端機使用的連線指令 `ssh root@< IP Address_記得做替換>`，然後按 Enter。
  4. 系統會詢問你要將設定儲存在哪個 SSH configuration 檔案，通常選擇第一個預設路徑即可
  5. 接著再次按下 F1，選擇 Remote-SSH: Connect to Host...，此時選單中會出現 <IP Address>，點擊它進行連線
  6. 如果系統詢問遠端作業系統類型，請選擇 Linux
  7. 在上方跳出的提示框中輸入 VM 密碼: `kw4Xtz4trmthQbgr`

# VS Code Setting Up

出現 SSH: <IP Address>即成功



點選左側的「檔案總管 (Explorer)」圖示，  
會看到一個「開啟資料夾 (Open Folder)」的按鈕。



點擊後，  
上方會跳出遠端 Ubuntu 系統的路徑  
選單(預設在/root/)，按下確定

# xApp Development (KPM\_Capture)

檔案路徑：`openairinterface5g/openair2/E2AP/flexric/examples/xApp/c/capture\_kpm/xapp\_kpm\_capture.c`  
( 建立新檔案，以官方範例 `monitor/xapp\_kpm\_moni.c` 為模板，訂閱流程不變，把列印改成 CSV 擷取)

```
openairinterface5g > openair2 > E2AP > flexric > examples > xApp > c > capture_kpm > xapp_kpm_capture.c
1  #include "../../src/xApp/e42_xapp_api.h"
2  #include "../../src/util/alg_ds/alg_defer.h"
3  #include "../../src/util/alg_ds/ds/lock_guard/lock_guard.h"
4  #include "../../src/util/e.h"
5
6  #include <stdlib.h>
7  #include <stdio.h>
8  #include <string.h>
9  #include <unistd.h>
10 #include <pthread.h>
11
12 #define OBS_NAME_BUF_VOL    "DRB.PdcpSduVolumeDL"
13 #define OBS_NAME_RLC_DELAY "DRB.RlcSduDelayDL"
14 #define OBS_NAME_THP_DL    "DRB.UETHpDL"
15 #define OBS_NAME_PRB_DL    "RRU.PrbTotDL"
16
17 static uint64_t const period_ms = 1000;
18
19 static pthread_mutex_t mtx;
20 static FILE* csv_fp;
21
22 typedef struct {
23     double v[4];
24     bool got[4];
25 } obs_row_t;
26
27 static
28 double meas_record_to_double(meas_record_lst_t const* r)
29 {
30     if (r->value == INTEGER_MEAS_VALUE)
31         return (double)r->int_val;
32     if (r->value == REAL_MEAS_VALUE)
33         return r->real_val;
34     return 0.0;
35 }
```



# xApp Development (KPM\_Capture)

程式碼解釋：

這支程式的主要功能是 gNB 訂閱並擷取特定的關鍵效能指標 (KPM)，然後準備將這些數據寫入 CSV 檔案中進行紀錄。

- 定義擷取指標 (Lines 12-15)

OBS\_NAME\_BUF\_VOL (DRB.PdcpSduVolumeDL)：PDCP 層的下行資料傳輸量。

OBS\_NAME\_RLC\_DELAY (DRB.RlcSduDelayDI)：RLC 層的下行延遲時間。

OBS\_NAME\_THP\_DL (DRB.UEThpDI)：使用者設備 (UE) 的下行吞吐量。

OBS\_NAME\_PRB\_DL (RRU.PrbTotDI)：下行物理資源塊 (PRB) 的總使用量。

- 型別轉換函式 (Lines 27-35)

meas\_record\_to\_double 是一個輔助函式。由於基站透過 E2 介面回傳的數據，其內部資料型態可能是整數或是浮點數。這個函式的作用是自動判斷回傳值的型態，並統一將其轉型為雙精度浮點數，以便後續一致地寫入 CSV 檔案進行分析。如果遇到無法辨識的型態，則回傳 0.0。

這段程式碼主要完成了 xApp 的環境設定與變數初始化。

# xApp Development (KPM\_Capture)

```
36
37 static
38 uint64_t get_ue_id_num(ue_id_e2sm_t const ue_id)
39 {
40     switch (ue_id.type) {
41         case GNB_UE_ID_E2SM:
42             if (ue_id.gnb.ran_ue_id != NULL)
43                 return *ue_id.gnb.ran_ue_id;
44             return ue_id.gnb.amf_ue_ngap_id;
45         case GNB_DU_UE_ID_E2SM:
46             if (ue_id.gnb_du.ran_ue_id != NULL)
47                 return *ue_id.gnb_du.ran_ue_id;
48             return ue_id.gnb_du.gnb_cu_ue_f1ap;
49         case GNB_CU_UP_UE_ID_E2SM:
50             if (ue_id.gnb_cu_up.ran_ue_id != NULL)
51                 return *ue_id.gnb_cu_up.ran_ue_id;
52             return ue_id.gnb_cu_up.gnb_cu_cp_ue_e1ap;
53         default:
54             return 0;
55     }
56 }
```

# xApp Development (KPM\_Capture)

程式碼解釋:

定義了一個名為 `get_ue_id_num` 的靜態函式，主要功能是從 O-RAN 的 E2 服務模型 ( E2SM ) 資料結構中，萃取出一個能唯一代表使用者設備 ( UE ) 的 64 位元整數 ID。

- 判斷節點類型 (Line 40)

`switch (ue_id.type):` 透過 `switch` 語法，根據回報該資訊的 O-RAN 節點類型，決定要讀取結構體中的哪一個特定欄位。

- 各節點類型的 ID 提取邏輯 (Lines 41-55)

程式碼針對三種不同的 5G 基地台架構情況進行處理

`case GNB_UE_ID_E2SM:` (完整的 gNB 節點)

`case GNB_DU_UE_ID_E2SM:` (分佈單元 DU 節點)

`case GNB_CU_UP_UE_ID_E2SM:` (集中單元使用者面 CU-UP 節點)

總結來說這是一個「UE ID 正規化」的輔助函式。無論 xApp 是從基地台的哪一個分離元件 ( CU 或 DU ) 收到測量報告，這個函式都能確保程式可以順利抓到一個有效的 UE ID，以便後續將各種效能指標 ( 如吞吐量、PRB 資源塊使用量 ) 與特定的手機或設備正確對應起來。

# xApp Development (KPM\_Capture)

```
57
58 static
59 void capture_kpm_measurements(kpm_ind_msg_format_1_t const* msg_frm_1, uint64_t ue_id_num)
60 {
61     assert(msg_frm_1->meas_info_lst_len > 0 && "Cannot correctly capture measurements");
62
63     for (size_t j = 0; j < msg_frm_1->meas_data_lst_len; j++) {
64         meas_data_lst_t const data_item = msg_frm_1->meas_data_lst[j];
65
66         obs_row_t row = {0};
67
68         for (size_t z = 0; z < data_item.meas_record_len; z++) {
69             meas_type_t const meas_type = msg_frm_1->meas_info_lst[z].meas_type;
70             meas_record_lst_t const record_item = data_item.meas_record_lst[z];
71
72             if (meas_type.type != NAME_MEAS_TYPE)
73                 continue;
74
75             char* name = cp_ba_to_str(meas_type.name);
76             double const val = meas_record_to_double(&record_item);
77
78             if (strcmp(name, OBS_NAME_BUF_VOL) == 0) {
79                 row.v[0] = val;
80                 row.got[0] = true;
81             } else if (strcmp(name, OBS_NAME_RLC_DELAY) == 0) {
82                 row.v[1] = val;
83                 row.got[1] = true;
84             } else if (strcmp(name, OBS_NAME_THP_DL) == 0) {
85                 row.v[2] = val;
86                 row.got[2] = true;
87             } else if (strcmp(name, OBS_NAME_PRB_DL) == 0) {
88                 row.v[3] = val;
89                 row.got[3] = true;
90             }
91         }
```

# xApp Development (KPM\_Capture)

程式碼解釋:

解析從 O-RAN 基站端傳來的 E2 KPM Format 1 訊息，並將裡面夾帶的原始測量數據，精準地對應到我們之前宣告的陣列中。

- 函式宣告 (Lines 58-61)

函式參數：`capture_kpm_measurements` 接收兩個參數：一個是指向 KPM 訊息的唯讀指標 `msg_frm_1`，另一個是剛剛透過輔助函式萃取出來的手機識別碼 `ue_id_num`。

- 外層迴圈：遍歷資料集 (Lines 63-66)

外層的 `for` 迴圈負責一筆一筆讀取。

- 內層迴圈：萃取與轉型 (Lines 68-76)

針對每一筆資料集，內層迴圈會將裡面的每一項具體測量紀錄抓出來。

# xApp Development (KPM\_Capture)

程式碼解釋:

- 數據匹配與歸檔 (Lines 78-90)

一連串的 if-else if 條件判斷式，利用 strcmp (字串比較函式) 來比對剛剛抓下來的指標名稱，是否符合在程式碼最前面定義的四個目標 KPM，分別處理延遲 (row.v[1]) 和吞吐量 (row.v[2])。

- 關鍵指標 (Lines 87-89)：

這裡處理的是 OBS\_NAME\_PRB\_DL。這將數值存入 row.v[3]。

# xApp Development (KPM\_Capture)

```
91
92     free(name);
93
94     if (data_item.incomplete_flag && *data_item.incomplete_flag == TRUE_ENUM_VALUE)
95     | printf("Measurement Record not reliable\n");
96 }
97
98 if (row.got[0] || row.got[1] || row.got[2] || row.got[3]) {
99     lock_guard(&mtx);
100     fprintf(csv_fp, "embb,%.6f,%.6f,%.6f,%.6f,%lu\n", row.v[0], row.v[1], row.v[2], row.v[3], ue_id_num);
101     fflush(csv_fp);
102     printf("[capture] ue=%lu embb obs=[%.3f, %.3f, %.3f, %.3f]\n", ue_id_num, row.v[0], row.v[1], row.v[2], row.v[3]);
103 }
104 }
105 }
106
107 static
108 void sm_cb_kpm(sm_ag_if_rd_t const* rd)
109 {
110     assert(rd != NULL);
111     assert(rd->type == INDICATION_MSG_AGENT_IF_ANS_V0);
112     assert(rd->ind.type == KPM_STATS_V3_0);
113
114     kpm_ind_data_t const* ind = &rd->ind.kpm.ind;
115     kpm_ind_msg_format_3_t const* msg_frm_3 = &ind->msg.frm_3;
116
117     for (size_t i = 0; i < msg_frm_3->ue_meas_report_lst_len; i++) {
118         uint64_t const ue_id_num = get_ue_id_num(msg_frm_3->meas_report_per_ue[i].ue_meas_report_lst);
119         capture_kpm_measurements(&msg_frm_3->meas_report_per_ue[i].ind_msg_format_1, ue_id_num);
120     }
121 }
```

# xApp Development (KPM\_Capture)

程式碼解釋:

- 寫入 CSV 檔案與終端機輸出 (Lines 98-105)
  - ✓ 條件判斷 (Line 98)：確認剛剛定義的四個指標 (row.got[0] 到 [3]) 至少有一個被成功擷取，才會進入寫入流程，避免寫入全空的無效資料。
  - ✓ 執行緒安全 (Line 99)：呼叫 `lock_guard(&mtx)`。由於 xApp 可能會同時處理來自多個 UE 或多個節點的並行報告，這個互斥鎖能保護檔案寫入過程，避免資料交錯或覆蓋。
  - ✓ 檔案儲存與輸出 (Lines 100-102)。
- KPM 服務模型回呼函式 (Lines 107-121)

`sm_cb_kpm` 函式：當 FlexRIC 收到來自 gNB 的 E2 Indication 訊息時，底層架構會自動觸發這個 Callback 函式。

迴圈處理每個 UE (Lines 117-120)：使用 for 迴圈遍歷這個 UE 報告列表，針對每一個連接的手機執行以下動作：

  - ：呼叫前面的程式碼圖提到的 `get_ue_id_num`，取得該手機統一的 UE ID。將該 UE 的 Format 1 具體測量數據與 UE ID 一併傳遞給 `capture_kpm_measurements` 函式進行數據的過濾、轉型與存檔。



# xApp Development (KPM\_Capture)

```
122
123 static
124 test_info_lst_t filter_predicate(test_cond_type_e type, test_cond_e cond, int value)
125 {
126     test_info_lst_t dst = {0};
127
128     dst.test_cond_type = type;
129     dst.S_NSSAI = TRUE_TEST_COND_TYPE;
130
131     dst.test_cond = calloc(1, sizeof(test_cond_e));
132     assert(dst.test_cond != NULL && "Memory exhausted");
133     *dst.test_cond = cond;
134
135     dst.test_cond_value = calloc(1, sizeof(test_cond_value_t));
136     assert(dst.test_cond_value != NULL && "Memory exhausted");
137     dst.test_cond_value->type = OCTET_STRING_TEST_COND_VALUE;
138
139     dst.test_cond_value->octet_string_value = calloc(1, sizeof(byte_array_t));
140     assert(dst.test_cond_value->octet_string_value != NULL && "Memory exhausted");
141     const size_t len_nssai = 1;
142     dst.test_cond_value->octet_string_value->len = len_nssai;
143     dst.test_cond_value->octet_string_value->buf = calloc(len_nssai, sizeof(uint8_t));
144     assert(dst.test_cond_value->octet_string_value->buf != NULL && "Memory exhausted");
145     dst.test_cond_value->octet_string_value->buf[0] = value;
146
147     return dst;
148 }
149
150 static
151 label_info_lst_t fill_kpm_label(void)
152 {
153     label_info_lst_t label_item = {0};
154
155     label_item.noLabel = calloc(1, sizeof(enum_value_e));
156     *label_item.noLabel = TRUE_ENUM_VALUE;
157
158     return label_item;
159 }
```

# xApp Development (KPM\_Capture)

程式碼解釋:

協助建構 E2 介面的「訂閱過濾條件」與「標籤設定」。

在 xApp 向 O-RAN 基站要求定期回報數據之前，必須先打包一份 Subscription Request，這份請求需要明確定義「我要收集什麼特定條件下的數據」。這兩個函式就是在準備這些細部的參數結構。

- filter\_predicate 函式 (Lines 123-148)

這個函式的功用是建立一個「資料過濾器」，用來告訴基站只回報符合特定條件的測量數據。

- ✓ 過濾目標 (Line 129)：

- ✓ 寫入過濾數值 (Lines 142-145)：程式定義了一個長度為 1 (`len_nssai = 1`) 的位元組陣列，並將函式傳入的參數 `value` 寫入陣列的第一個位置 `buf[0]`。這個 `value` 就是想監控的那個特定網路切片 ID。

- fill\_kpm\_label 函式 (Lines 150-159)

這個函式負責建立 KPM 測量數據的「Label」。

在 O-RAN E2SM-KPM 規範中，基站回傳的數據可以附加各種標籤，例如標示這筆數據屬於哪一個 QoS 流程、哪一個頻段，或是哪一個電信業者。

# xApp Development (KPM\_Capture)

```
160
161 static
162 kpm_act_def_format_1_t fill_act_def_frm_1(ric_report_style_item_t const* report_item)
163 {
164     assert(report_item != NULL);
165
166     kpm_act_def_format_1_t ad_frm_1 = {0};
167
168     size_t const sz = report_item->meas_info_for_action_lst_len;
169
170     ad_frm_1.meas_info_lst_len = sz;
171     ad_frm_1.meas_info_lst = calloc(sz, sizeof(meas_info_format_1_lst_t));
172     assert(ad_frm_1.meas_info_lst != NULL && "Memory exhausted");
173
174     for (size_t i = 0; i < sz; i++) {
175         meas_info_format_1_lst_t* meas_item = &ad_frm_1.meas_info_lst[i];
176         meas_item->meas_type.type = NAME_MEAS_TYPE;
177         meas_item->meas_type.name = copy_byte_array(report_item->meas_info_for_action_lst[i].name);
178
179         meas_item->label_info_lst_len = 1;
180         meas_item->label_info_lst = calloc(1, sizeof(label_info_lst_t));
181         meas_item->label_info_lst[0] = fill_kpm_label();
182     }
183
184     ad_frm_1.gran_period_ms = period_ms;
185     ad_frm_1.cell_global_id = NULL;
186
187     #if defined KPM_V2_03 || defined KPM_V3_00
188         ad_frm_1.meas_bin_range_info_lst_len = 0;
189         ad_frm_1.meas_bin_info_lst = NULL;
190     #endif
191
192     return ad_frm_1;
193 }
```

# xApp Development (KPM\_Capture)

程式碼解釋:

`fill_act_def_frm_1` 函式，是負責建構 O-RAN E2 介面中非常關鍵的「動作定義」。

當 xApp 向基地台 (gNB) 發送訂閱請求時，除了要指明訂閱的類型，還必須透過 Action Definition 明確告訴基地台：「請按照這個格式，把這幾個特定的指標數據整理好傳給我」。

- 參數接收 (Lines 162-172)

傳入參數 (Line 162)：函式接收一個名為 `report_item` 的唯讀指標。這個變數裡面包含了基地台目前支援，且 xApp 打算收集的指標清單（例如前面定義的 PRB 總量、緩衝區傳輸量等）。

- 迴圈設定測量指標 (Lines 174-182)

一個 for 迴圈，用來將清單上的每一個測量指標進行標準化設定：

- ✓ 指定名稱類型 (Line 176)：NAME\_MEAS\_TYPE 告訴基地台，我們是透過「字串名稱」來指定測量項目，而不是用數字 ID。
- ✓ 複製指標名稱 (Line 177)：將 `report_item` 中的指標名稱字串 copy 到我們準備發送的請求結構中。
- ✓ 套用空白標籤 (Lines 179-181)：呼叫了前面出現的 `fill_kpm_label()` 函式。表示我們要求基地台回傳這些數據（如 PRB 數量）的總和原始值即可。

# xApp Development (KPM\_Capture)

```
194
195 static
196 kpm_act_def_t fill_report_style_4(ric_report_style_item_t const* report_item)
197 {
198     assert(report_item != NULL);
199     assert(report_item->act_def_format_type == FORMAT_4_ACTION_DEFINITION);
200
201     kpm_act_def_t act_def = {.type = FORMAT_4_ACTION_DEFINITION};
202
203     act_def.frm_4.matching_cond_lst_len = 1;
204     act_def.frm_4.matching_cond_lst = calloc(act_def.frm_4.matching_cond_lst_len, sizeof(matching_condition_format_4_t));
205     assert(act_def.frm_4.matching_cond_lst != NULL && "Memory exhausted");
206
207     test_cond_type_e const type = S_NSSAI_TEST_COND_TYPE;
208     test_cond_e const condition = EQUAL_TEST_COND;
209     int const value = 1;
210     act_def.frm_4.matching_cond_lst[0].test_info_lst = filter_predicate(type, condition, value);
211
212     act_def.frm_4.action_def_format_1 = fill_act_def_frm_1(report_item);
213
214     return act_def;
215 }
216
217 typedef kpm_act_def_t (*fill_kpm_act_def)(ric_report_style_item_t const* report_item);
218
219 static
220 fill_kpm_act_def get_kpm_act_def[END_RIC_SERVICE_REPORT] = {
221     NULL,
222     NULL,
223     NULL,
224     fill_report_style_4,
225     NULL,
226 };
```

程式碼解釋:

將前面定義好的「資料過濾條件」與「動作定義格式」組裝在一起，並建立了一個發送請求的對應表。

- 實作 Report Style 4 的封裝函式 (Lines 195-215)

fill\_report\_style\_4 這個函式是把設定檔打包成基地台看得懂的格式 4 結構。

設定具體的過濾條件 (Lines 207-210)：

- ✓ type = S\_NSSAI\_TEST\_COND\_TYPE：指定過濾目標為 5G 網路切片。
- ✓ condition = EQUAL\_TEST\_COND：設定邏輯為「等於」。
- ✓ value = 1：指定切片 ID 為 1。

接著呼叫前面提到的 filter\_predicate 函式，將「只收集切片 ID 為 1 的數據」這個規則寫入設定中。

綁定測量內容 (Line 212)：呼叫前面的 fill\_act\_def\_frm\_1 函式。針對上述過濾出來的切片 1 流量，請套用 Format 1 的規則，把緩衝區傳輸量、延遲、吞吐量，以及最重要的下行物理資源塊 (PRB) 總使用量抓出來

- 建立函式指標對應表 (Lines 217-226)

「分派表 (Dispatch Table)」設計模式。

當主程式 E2 agent 需要發送訂閱請求時，它會直接來查這個表，發現只有 Style 4 有實作，就會自動去呼叫上面的封裝邏輯，完成整個向基地台訂閱資料的流程。

# xApp Development (KPM\_Capture)

```
227
228 static
229 kpm_sub_data_t gen_kpm_subs(kpm_ran_function_def_t const* ran_func)
230 {
231     assert(ran_func != NULL);
232     assert(ran_func->ric_event_trigger_style_list != NULL);
233
234     kpm_sub_data_t kpm_sub = {0};
235
236     assert(ran_func->ric_event_trigger_style_list[0].format_type == FORMAT_1_RIC_EVENT_TRIGGER);
237     kpm_sub.ev_trg_def.type = FORMAT_1_RIC_EVENT_TRIGGER;
238     kpm_sub.ev_trg_def.kpm_ric_event_trigger_format_1.report_period_ms = period_ms;
239
240     kpm_sub.sz_ad = 1;
241     kpm_sub.ad = calloc(kpm_sub.sz_ad, sizeof(kpm_act_def_t));
242     assert(kpm_sub.ad != NULL && "Memory exhausted");
243
244     ric_report_style_item_t* const report_item = &ran_func->ric_report_style_list[0];
245     ric_service_report_e const report_style_type = report_item->report_style_type;
246     *kpm_sub.ad = get_kpm_act_def[report_style_type](report_item);
247
248     return kpm_sub;
249 }
250
251 static
252 bool eq_sm(sm_ran_function_t const* elem, int const id)
253 {
254     return elem->id == id;
255 }
```

# xApp Development (KPM\_Capture)

程式碼解釋：

組裝成一份完整的「KPM 訂閱請求 (Subscription Request) 封包資料」。

- 產生完整訂閱資料：gen\_kpm\_subs (Lines 227-249)

這個函式負責將「觸發條件」與「動作定義」打包進一個名為 kpm\_sub\_data\_t 的結構體中，這是準備透過 E2 介面送給基地台的最終資料格式。

設定事件觸發條件 (Lines 236-238)：

- ✓ 這裡定義了基地台應該「何時」發送報告。
  - ✓ FORMAT\_1\_RIC\_EVENT\_TRIGGER 代表這是一個週期性的觸發器。
  - ✓ report\_period\_ms = period\_ms：將回報週期綁定為先前設定的 1000 毫秒。這意味著 xApp 將會以每秒一次的頻率，精確地收集基地台當下的狀態。
- 服務模型比對工具：eq\_sm (Lines 251-255)



# xApp Development (KPM\_Capture)

```
256
257 static
258 size_t find_sm_idx(sm_ran_function_t* rf, size_t sz, bool (*f)(sm_ran_function_t const*, int const), int const id)
259 {
260     for (size_t i = 0; i < sz; i++) {
261         if (f(&rf[i], id))
262             return i;
263     }
264
265     assert(0 != 0 && "SM ID could not be found in the RAN Function List");
266 }
```

# xApp Development (KPM\_Capture)

```
267
268 int main(int argc, char* argv[])
269 {
270     fr_args_t args = init_fr_args(argc, argv);
271
272     char const* csv_path = getenv("KPM_CAPTURE_CSV");
273     if (csv_path == NULL)
274         csv_path = "kpm_capture.csv";
275
276     csv_fp = fopen(csv_path, "w");
277     if (csv_fp == NULL) {
278         perror("fopen csv_path");
279         exit(EXIT_FAILURE);
280     }
281     fprintf(csv_fp, "slice_name,obs_0,obs_1,obs_2,obs_3,ue_id\n");
282     fflush(csv_fp);
283     printf("[capture] writing to %s\n", csv_path);
284
285     init_xapp_api(&args);
286     sleep(1);
287
288     e2_node_arr_xapp_t nodes = e2_nodes_xapp_api();
289     defer({ free_e2_node_arr_xapp(&nodes); });
290
291     assert(nodes.len > 0);
292
293     printf("Connected E2 nodes = %d\n", nodes.len);
294
295     pthread_mutexattr_t attr = {0};
296     int rc = pthread_mutex_init(&mtx, &attr);
297     assert(rc == 0);
298
299     sm_ans_xapp_t* hndl = calloc(nodes.len, sizeof(sm_ans_xapp_t));
300     assert(hndl != NULL);
301
```

# xApp Development (KPM\_Capture)

程式碼解釋:

這支 xApp 的核心進入點：main 函式的前半段。這段程式碼負責整個 xApp 的啟動與基礎設施初始化，包含了參數讀取、CSV 檔案建置、與 FlexRIC 核心建立連線，以及多執行緒環境的準備。

- 參數解析與 CSV 輸出準備 (Lines 270-283)

為了準備記錄接下來從基站收集到的數據。

- ✓ 讀取啟動參數 (Line 270)：init\_fr\_args 負責處理命令列輸入的參數，並儲存到 args 結構中。
- ✓ 設定檔案路徑 (Lines 272-274)：程式會先檢查系統中是否有設定 KPM\_CAPTURE\_CSV 環境變數。如果沒有，就會預設將資料輸出到當前目錄下的 kpm\_capture.csv。
- ✓ 檔案開啟與錯誤處理 (Lines 276-280)：以寫入模式 ("w") 開啟 CSV 檔案。若因為權限或目錄不存在導致開啟失敗，程式會印出錯誤並立即以失敗狀態 (EXIT\_FAILURE) 終止。

寫入資料 header (Lines 281-283)：檔案成功建立後，寫入標頭 slice\_name,obs\_0,obs\_1,obs\_2,obs\_3,ue\_id。

- xApp 註冊與節點探索 (Lines 285-293)

啟動了 xApp 與底層 O-RAN 架構的互動。獲取連線節點 (Lines 288-293)：

# xApp Development (KPM\_Capture)

```
301
302 ///////////////
303 // START KPM
304 ///////////////
305 int const KPM_ran_function = 2;
306
307 for (size_t i = 0; i < nodes.len; ++i) {
308     e2_node_connected_xapp_t* n = &nodes.n[i];
309
310     size_t const idx = find_sm_idx(n->rf, n->len_rf, eq_sm, KPM_ran_function);
311     assert(n->rf[idx].defn.type == KPM_RAN_FUNC_DEF_E && "KPM is not the received RAN Function");
312     if (n->rf[idx].defn.kpm.ric_report_style_list != NULL) {
313         kpm_sub_data_t kpm_sub = gen_kpm_subs(&n->rf[idx].defn.kpm);
314
315         hndl[i] = report_sm_xapp_api(&n->id, KPM_ran_function, &kpm_sub, sm_cb_kpm);
316         assert(hndl[i].success == true);
317
318         free_kpm_sub_data(&kpm_sub);
319     }
320 }
321 ///////////////
322 // END KPM
323 ///////////////
324
325 xapp_wait_end_api();
326
327 for (int i = 0; i < nodes.len; ++i) {
328     if (hndl[i].success == true)
329         rm_report_sm_xapp_api(hndl[i].u.handle);
330 }
331 free(hndl);
332
333 fclose(csv_fp);
334
335 while (try_stop_xapp_api() == false)
336     usleep(1000);
337
338 printf("xapp_kpm_capture stopped SUCCESSFULLY\n");
339
340 return 0;
341 }
342
```

# xApp Development (KPM\_Capture)

程式碼解釋:

正式發送訂閱請求

- 發送 KPM 訂閱請求 (Lines 301-323)

尋找與驗證服務模型 (Lines 310-311)：呼叫 `find_sm_idx` 函式

生成訂閱資料 (Line 313)：呼叫 `gen_kpm_subs`，將過濾切片 1 與每秒回報一次 Format 1 的設定，打包成最終的 `kpm_sub` 結構。

正式發送請求 (Line 315)：`report_sm_xapp_api` 是資料擷取階段的 API。它向底層 agent 下達指令，將準備好的訂閱請求發送給特定的基地台。

最後一個參數 `sm_cb_kpm` (Callback function)。當基地台接受訂閱，並開始每秒送出 PRB、吞吐量等數據時，xApp 就會在背景自動觸發這個函式來接收並處理這些數據。

- 進入等待監聽狀態 (Line 325)

阻擋主執行緒 (Line 325)：`xapp_wait_end_api()` 會讓 main 函式停在這裡等待。這時程式進入了「常駐監聽模式」。

在等待的期間，一旦基地台送來數據，背景的執行緒就會不斷呼叫剛剛註冊的 `sm_cb_kpm` 結果一行一行寫入 CSV 檔案中。直到按下 Ctrl+C 發出終止訊號。

# **xApp Development (KPM\_Capture)**

## **--測試結果**

# xApp Development (KPM\_Capture)

- **Compile**

## **Step 1:編譯 FlexRIC**

```
cd ~/openairinterface5g/openair2/E2AP/flexric/build
```

```
cmake -G Ninja -DCMAKE_BUILD_TYPE=Release -DKPM_VERSION=KPM_V3_00 -DE2AP_VERSION=E2AP_V2 ..
```

```
ninja
```

```
sudo ninja install
```

## **Step 2:編譯支援 E2 Agent 的 OAI RAN**

```
cd ~/openairinterface5g/cmake_targets
```

# (注意:該指令合併為完整的一行，不可斷行，複製貼上時須小心)

```
sudo ./build_oai --gNB --nrUE --build-e2 --ninja -w USRP -C --cmake-opt -DE2AP_VERSION=E2AP_V2 --cmake-opt -DKPM_VERSION=KPM_V3_00 --cmake-opt -DCMAKE_BUILD_TYPE=Release
```

# xApp Development (KPM\_Capture)

- Run OAI CN5G , OAI gNB, OAI UE, nearRT-RIC

## Step 1:Run OAI CN5G

```
cd ~/oai-cn5g
```

```
# 讀取目錄下的 docker-compose.yaml 檔案，自動建立並啟動 5G 核心網所需的所有容器  
docker compose up -d
```

```
# 驗證容器運行狀態  
docker ps -a
```

## Step 2:Run UE pipeline ( nearRT-RIC → gNB → UE → xApp，內含 3 輪 idle/loaded 流量，約需 3~4 min )

```
# 執行過程會依序印出 `[0/6]` ~ `[6/6]` 的階段訊息  
# 正常結束會看到：  
All done. Capture file: /root/xapp-dev/capture/kpm_capture.csv
```

```
/root/xapp-dev/scripts/run_pipeline.sh
```

## Step 3：停止所有元件 ( gNB/UE/RIC/iperf3 server )

```
/root/xapp-dev/scripts/stop_pipeline.sh
```



# xApp Development (KPM\_Capture)

```
root@workshop-test:~/oai-cn5g# /root/xapp-dev/scripts/run_pipeline.sh
[0/6] Ensuring gNB<->AMF host-side bridge IP is present (docker doesn't persist this across host reboots)...
      adding missing 192.168.70.151/26 on oai-cn5g...
[1/6] Starting nearRT-RIC...
nearRT-RIC up (pid 28944)
[2/6] Starting gNB (nr-softmodem, rfsim + E2 agent, pinned to cores 1-3)...
gNB up (pid 29034), waiting for rfsim server + E2 setup...
[3/6] Starting UE (nr-uesoftmodem, rfsim, pinned to cores 1-3, up to 3 registration attempts)...
      attempt 1/3...
      UE process up (pid 30048), waiting for RRC/NAS registration + PDU session...
      attempt 1: succeeded
      UE PDU session IP = 10.0.0.2
      Registration complete -- releasing gNB/UE CPU pin (all cores available) to maximize DL throughput for the loaded phases...
[4/6] Starting iperf3 server on UE host (port 5202, no -B needed -- only the UPF/GTP path reaches 10.0.0.2)...
iperf3 server up (pid 32120)
[5/6] Starting xapp_kpm_capture (duration 120s)...
xapp_kpm_capture up (pid 32401), writing to /root/xapp-dev/capture/kpm_capture.csv
[6/6] Driving phased DL traffic (idle/loaded x3) from oai-ext-dn -> 10.0.0.2:5202 for ~120s...
      idle phase 1 (15s, no traffic -> obs_0~0 -> eco-routed rows)...
      loaded phase 1 (25s, iperf3 client in oai-ext-dn -> UE, TCP, 4 parallel streams)...
      idle phase 2 (15s)...
      loaded phase 2 (25s)...
      idle phase 3 (15s)...
      loaded phase 3 (25s)...
      Traffic phases done, waiting for xapp_kpm_capture (pid 32401) to finish its capture window...
      All done. Capture file: /root/xapp-dev/capture/kpm_capture.csv
      Stop remaining components (RIC/gNB/UE/iperf3 server) with: /root/xapp-dev/scripts/stop_pipeline.sh
root@workshop-test:~/oai-cn5g#
```

執行 run\_pipeline.sh 腳本之後的結果

# xApp Development (KPM\_Capture)

```
xapp_kpm_capture.c U kpm_capture.csv X
xapp-dev > capture > kpm_capture.csv
-----
17 embb,0.000000,0.000000,0.000000,0.000000,1
18 embb,0.000000,0.000000,0.000000,0.000000,1
19 embb,0.000000,0.000000,0.000000,0.000000,1
20 embb,3.000000,85111.626437,3331.448000,20.000000,1
21 embb,12.000000,91300.457399,11537.496000,53.000000,1
22 embb,13.000000,1394.906780,13858.832000,29.000000,1
23 embb,17.000000,3982.184713,17783.568000,17.000000,1
24 embb,22.000000,10673.929936,22809.768000,23.000000,1
25 embb,16.000000,1595.719298,16055.512000,13.000000,1
26 embb,14.000000,6012.434286,14788.416000,10.000000,1
27 embb,21.000000,5353.109290,22327.496000,19.000000,1
28 embb,17.000000,2111.698276,16504.672000,13.000000,1
29 embb,12.000000,2477.887218,12685.824000,9.000000,1
30 embb,22.000000,7569.429630,22022.384000,19.000000,1
31 embb,20.000000,7482.069652,21525.856000,19.000000,1
32 embb,16.000000,1998.484375,16521.016000,12.000000,1
33 embb,20.000000,2509.236515,19956.608000,16.000000,1
34 embb,22.000000,5091.228972,22852.144000,20.000000,1
35 embb,24.000000,5128.674419,24679.416000,24.000000,1
36 embb,20.000000,3247.442105,20481.384000,16.000000,1
37 embb,22.000000,4918.056522,23086.592000,20.000000,1
38 embb,22.000000,6749.260870,22387.944000,19.000000,1
39 embb,21.000000,3305.487437,21555.216000,17.000000,1
40 embb,15.000000,16779.034483,15257.744000,14.000000,1
41 embb,20.000000,7203.862559,20530.344000,18.000000,1
42 embb,23.000000,5987.429150,23069.904000,19.000000,1
43 embb,23.000000,1901.029851,23681.720000,20.000000,1
44 embb,19.000000,4936.632653,19855.448000,16.000000,1
45 embb,16.000000,75.000000,16629.456000,12.000000,1
46 embb,0.000000,0.000000,2.440000,0.000000,1
47 embb,0.000000,0.000000,0.000000,0.000000,1
```

kpm\_capture.csv 結果

# 本機驗證模型推論

# 本機驗證模型推論結果

下載推論模型:

<https://drive.google.com/drive/folders/16e5HxWuCmSuii6j0SarGc7XFaLVM456g?usp=sharing>

我的雲端硬碟 > model

類型 使用者 上次修改日期 來源

| 名稱                | 擁有者 | 修改日期     | 檔案大小    | 排序 |
|-------------------|-----|----------|---------|----|
| app.py            | 我   | 下午3:26 我 | 4 KB    |    |
| model.pt          | 我   | 4月23日 我  | 30 KB   |    |
| README_model.md   | 我   | 5月18日 我  | 5 KB    |    |
| requirements.txt  | 我   | 5月18日 我  | 42 個位元組 |    |
| test_samples.csv  | 我   | 5月18日 我  | 31 KB   |    |
| verify_capture.py | 我   | 8月5日 我   | 3 KB    |    |

下載至本機中 C:\Users\<username\_記得做替換>\Downloads 中

# 本機驗證模型推論結果

**Step 1：把擷取結果抓回本機（在本機的 powershell 中執行，不是在 SSH session 裡）**

```
scp root@<IP Address_記得做替換>:/root/xapp-dev/capture/kpm_capture.csv "C:\Users\<username_記得做替換>\Downloads\model\kpm_capture.csv"
```

**Step 2：驗證模型推論（在本機的 Downloads/model 資料夾內 powershell 執行，不是在 SSH session 裡）**

```
# 在本機的 Downloads/model/ 路徑內開一個 powershell，啟動 Flask 推論服務  
py -3 app.py
```

```
# 在本機的 Downloads/model/ 路徑內，再開另一個 powershell，把剛抓回來的 CSV 逐列送進 /predict  
py -3 verify_capture.py kpm_capture.csv
```

```
# 正常應該會看到 `eco-routed` 與 `ai-routed` 都是非 0（代表 Eco 節能路徑與 AI 神經網路推論路徑都有被真實資料觸發到），`errors` 為 0。
```

# 本機驗證模型推論結果

```
PS C:\Users\lindor\Downloads\模組D model> py -3 verify_capture.py kpm_capture.csv
[health] {"buffer_threshold":15.0,"status":"ok"}

[info] loaded 120 rows from kpm_capture.csv
[sample response] {
  "predictions": [
    {
      "action": 1,
      "slice_name": "embb",
      "source": "eco"
    },
    {
      "action": 1,
      "slice_name": "embb",
      "source": "eco"
    },
    {
      "action": 1,
      "slice_name": "embb",
      "source": "eco"
    }
  ]
}
```

```
=== Summary ===
total rows      : 120
eco-routed      : 57
ai-routed       : 63
errors          : 0
```

模型推論結果

# 本機驗證模型推論結果

觸發節能機制:

kpm\_capture.csv 裡每一列,都是 xApp 透過 E2SM-KPM 從真實跑在 rfsim 上的 gNB+UE 每秒擷取到的一筆真實 RAN 量測(緩衝量、延遲、吞吐量、PRB 使用率)。把這些真實資料餵進 model.pt,等於在問:「如果這是一個真的部署在 near-RT RIC 上的 DQN 資源分配 xApp,看到這個 embb 當下這樣的流量狀況,它會怎麼決策?」

- source: eco :那個當下切片的 obs\_0,即 DRB.PdcpSduVolumeDL 低於門檻(15) 直接回傳固定的**節能**動作。
- source: ai :緩衝量夠高, 才做無線資源分配決策。

# Thank You